

API Reference

This web interface runs on port :3000, however the system also provides a RESTful API on port :8000. Anything done in this web interface is based on this API.

The API accepts and returns only JSON encoded objects with no need for authentication. There are different ways to use it, one is with Python:

```
import requests
url = 'http://<ip>:8000/<Endpoint>'
headers = {
    "Content-type": "application/json"
}

res = requests.get(url, headers=headers)
print(res.text)
```

In each of the endpoints bellow it's provided the expected returns and a sample code to use it.
All code can be seen on the [GitHub](#) repository.

STATUS

GET /api

Returns a message if the API is up and running.

```
{
  "message": "API running."
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/api"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

GET /get-status

Returns a message with the current state which can be on of the following:

```
{
  // Ready to start a test
  "message": "Ready"
  // Already running a test
  "message": "Running a test"
  // Aborting the running test
  "message": "Aborting the test"
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-status"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

GET /get-current-parameters

If a test is running, returns an object with the parameters being used:

```
{
  // If a test is running
  "pressTime": 0
  "pressTime": 1
  "pressTime": 0
}
{
  // If no test is running
  "message": "Not running a test"
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-current-parameters"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

GET /get-logs

Returns a file with the logs of the last test done. Can be accessed by just opening on the browser.

RUN A TEST

POST /start

Starts a button test, with the given parameters or profile chosen. If a valid profile is given the other parameters are ignored. A request with no body starts the test with parameter nTimes equal to one an the others set to zero. Returns one of the following messages:

```
{
  // Status code 200 OK
  "message": "Test started."
  // Status code 304 Not Modified
  "message": "A Test is already running."
  // Status code 404 Not Found
  "message": "Profile not found"
  // Status code 422 Unprocessable Entity
  "detail": [
    ...
  ]
}
```

Sample python Code:

```
import requests
url = "http://localhost:8000/start"
headers = {
    "Content-type": "application/json"
}
body = {
    "pName": "Example"
    // Or define the parameters
    // "pressTime": 0
    // "nTimes": 1
    // "interval": 0
}

res = requests.post(url, headers=headers, json=body)
print(res.text)
```

GET /abort-test

If a test is running aborts it. The test will only abort after finishing the current actuation

```
{
  "message": "No test to abort"
  "message": "Test Aborted"
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/abort-test"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

TESTS DATA

GET /get-test-data

Returns a file a JSON file with the all tests done from newest to oldest. Using a browser you can open the link directly and a download should appear. Data has the format:

```
[
  {
    "id": 5,
    "button": "MPS",
    "success": 1,
    "error": "No error",
    "presses": 2,
    "parameters": "[0, 2, 1]",
    "force_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
    "time_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
    "date": "2024-12-30",
    "time": "11:45:43"
  },
  ...
]
```

GET /get-tests

Returns all tests done, from newest to oldest, with format:

```
{
  "tests": {
    "test": [
      {
        "id": 5,
        "button": "MPS",
        "success": 1,
        "error": "No error",
        "presses": 2,
        "parameters": "[0, 2, 1]",
        "force_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
        "time_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
        "date": "2024-12-30",
        "time": "11:45:43"
      },
      ...
    ]
  }
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-tests"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

GET /get-last-test

Returns only the last test done:

```
{
  "id": 5,
  "button": "MPS",
  "success": 1,
  "error": "No error",
  "presses": 2,
  "parameters": "[0, 2, 1]",
  "force_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
  "time_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
  "date": "2024-12-30",
  "time": "11:45:43"
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-last-test"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

GET /get-success

Returns the success of all tests done:

```
[
  1,
  1,
  0,
  ...
]
```

Sample python Code:

```
import requests

url = "http://localhost:8000/get-success"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

POST /get-tests-range

Returns a defined range of tests:

```
{
  "tests": {
    "test": [
      {
        "id": 5,
        "button": "MPS",
        "success": 1,
        "error": "No error",
        "presses": 2,
        "parameters": "[0, 2, 1]",
        "force_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
        "time_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
        "date": "2024-12-30",
        "time": "11:45:43"
      },
      ...
    ]
  }
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-tests-range"
headers = {
    "Content-type": "application/json"
}
body = {
    "size": 2,
    "offset": 3
}

res = requests.post(url, headers=headers, json=body)
print(res.text)
```

GET /get-count

Returns the total number of tests done:

```
{
  "count": 19
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-count"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

POST /get-by-id

Returns the test data with the same id:

```
{
  "id": 5,
  "button": "MPS",
  "success": 1,
  "error": "No error",
  "presses": 2,
  "parameters": "[0, 2, 1]",
  "force_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
  "time_val": "[[1, 2, 3, 4, 5], [1, 2, 3, 4, 5]]",
  "date": "2024-12-30",
  "time": "11:45:43"
}
```

Sample python Code:

```
import requests

url = "http://localhost:8000/get-by-id"
headers = {
    "Content-type": "application/json"
}
body = {
    "id": 5
}
res = requests.post(url, headers=headers, json=body)
print(res.text)
```

DELETE /delete-tests-before-date

Deletes all tests before the given date:

```
{
  "message": "Tests deleted"
}
```

Sample python Code:

```
import requests

url = "http://localhost:8000/delete-tests-before-date"
headers = {
    "Content-type": "application/json"
}
body = {
    "day": "2024-12-30"
}
res = requests.delete(url, headers=headers, json=body)
print(res.text)
```

PROFILES

GET /get-profiles

Returns all profiles:

```
{
  "profiles": {
    "profile": [
      {
        "id": 1,
        "pName": "Custom",
        "pressTime": 0,
        "nTimes": 0,
        "interval": 0
      },
      ...
    ]
  }
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/get-profiles"
headers = {
    "Content-type": "application/json"
}
res = requests.get(url, headers=headers)
print(res.text)
```

POST /add-profile

Add a new profile with given parameters and name. Returns one of the messages:

```
{
  "message": "Profile added to database."           // Status code 201 Created
  "message": "A profile with that name already exists." // Status code 400 Bad Request
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/add-profile"
headers = {
    "Content-type": "application/json"
}
body = {
    "pName": "Test Profile",
    "pressTime": 3,
    "nTimes": 2,
    "interval": 1
}

res = requests.post(url, headers=headers, json=body)
print(res.text)
```

DELETE /delete-profile

Deletes a profile given it's name. Returns on of the messages:

```
{
  "message": "Profile deleted from database."           // Status code 200 OK
  "message": "Profile name not found."                 // Status code 404 Not Found
}
```

Sample python code:

```
import requests

url = "http://localhost:8000/delete-profile"
headers = {
    "Content-type": "application/json"
}
body = {
    "pName": "Test Profile"
}

res = requests.delete(url, headers=headers, json=body)
print(res.text)
```